National University of Computer and Emerging Sciences

FAST-NUCES

Bachelor of Science in Data Science

Course Code: DS3003 & DS3004

Course Name: Data Warehousing & Business Intelligence

Semester: Fall 2026

Practical Project Report
Building and Analysing a Near-Real-Time and Periodic Data Warehouse for a Retail Chain

MESHJOIN-Based ETL Pipeline with Star Schema and OLAP Analysis

Course Instructor:	Muhammad Asif Naeem
Submission Date:	13 November 2026

Table of Contents

1. Introduction 3
2. Data Cleaning 4
3. Master Data Generation 5
4. Data Warehouse Design (Star Schema) 7
5. MESHJOIN Algorithm 9
6. Multi-Threading and Load Shedding 12
7. Pipeline Execution and Fact Loading 14
8. DW Analysis Queries 15
9. Conclusion 18

1. Introduction

This project is about building a Data Warehouse for a retail chain that sells products both online and through physical stores. The retailer generates two types of transaction data. Online sales arrive as a continuous stream and need to reach the Data Warehouse in near real time. Physical store sales are collected in batches every hour and loaded periodically.

The challenge is not just storing the data. The raw transactional records only contain basic fields like customer ID, product code, quantity, price, and time. Before any record can go into the Data Warehouse, it needs to be enriched with additional details, such as customer demographics and product category and supplier information. This process of joining streaming data with master data stored on disk is handled by a stream-relation join algorithm called MESHJOIN, which we implemented in Python.

The overall pipeline can be described in a few steps. First, the raw Excel file is cleaned. Then master data is generated for customers and products. The cleaned transactions are split into online and periodic streams. MESHJOIN runs in multiple threads to join the stream with master data and enrich each record. The enriched records are then loaded into the Data Warehouse, which is built using a star schema in MySQL. Finally, 20 OLAP business queries answer analytical questions about the retail chain's performance.

The entire implementation is written in Python for the ETL pipeline and MySQL for the database layer. All code files are organized inside the project directory as specified in the assignment requirements.

1.1 Project Directory Structure

```
final project/
├── Create-DW/
│   └── create_dw.sql           (database schema, dimensions, fact table, view)
├── Data/
│   ├── raw/
│   │   └── Online Retail.xlsx
│   ├── processed/
│   │   ├── cleaned_transactions.csv
│   │   ├── customer_master_data.csv
│   │   ├── product_master_data.csv
│   │   ├── online_sales_stream.csv
│   │   └── periodic_sales_staging.csv
│   └── README.md
├── Mesh-Join/
│   ├── data_preparation.py
│   ├── database.py
│   ├── meshjoin.py
│   ├── threads.py
│   ├── main.py
│   └── requirements.txt
├── Queries-DW/
```

```
│   └── analysis_queries.sql
└── README.md
```

2. Data Cleaning

The raw dataset is an Excel file called `Online Retail.xlsx`. It contains sales transactions of a UK-based online and physical retail chain recorded between December 2010 and December 2011.

Before any processing, the file is read as-is. We do not manually edit it. The cleaning is done entirely by the script `data_preparation.py`. The assignment says to keep a transaction only if all four of these conditions are true:

- Quantity is greater than zero (removes negative quantities, which are return adjustments)
- UnitPrice is greater than zero (removes test lines and manual corrections priced at zero)
- CustomerID is present and not null (keeps only identified customers)
- InvoiceNo does not start with the letter C (cancellation invoices start with C)

These four filters are applied one after another using pandas. Each condition removes a subset of records. The remaining records form the cleaned dataset that is used for all further steps.

2.1 Cleaning Results

Metric	Count
Transactions before cleaning	541,909
Transactions after cleaning	397,884
Transactions removed	144,025

So we removed about 26.6 percent of the original records. The cleaned dataset is saved as `cleaned_transactions.csv` inside the `Data/processed/` folder.

2.2 Cleaning Code

```
# keep only the not cancelled invoices
cleaned_data = dataset[~dataset["InvoiceNo"].astype(str).str.startswith("C")]

# keep only the valid quantities
cleaned_data = cleaned_data[cleaned_data["Quantity"] > 0]

# keep only the valid prices
cleaned_data = cleaned_data[cleaned_data["UnitPrice"] > 0]

# keep only customers with IDs
cleaned_data = cleaned_data[cleaned_data["CustomerID"].notna()]
```

3. Master Data Generation

The raw transactional data does not include customer demographics or product details beyond a stock code and description. The assignment requires us to generate this master data ourselves. The important requirement is that the data must be deterministic. That means every time the code runs, the exact same customer ID must get the exact same attributes. We cannot use random numbers for this.

To achieve this, we use an MD5-based hashing function called `stable(text, n)`:

```
def stable(text, n):
    value = hashlib.md5(str(text).encode("utf-8")).hexdigest()[:8]
    return int(value, 16) % n
```

This function takes any text input and returns a number between 0 and n-1 consistently. The same input always produces the same number, so the same customer ID always gets the same gender, age group, occupation, and so on.

3.1 Customer Master Data

We take all unique CustomerIDs from the cleaned dataset and generate the following attributes for each customer:

Attribute	Values	Hash Input
gender	Male, Female	stable(customer_id, 2)
age_group	0-17, 18-25, 26-35, 36-45, 46-50, 51-55, 55+	stable(customer_id + "age", 7)
occupation	Student, Engineer, Teacher, Executive, Clerk, Healthcare, Retail, Homemaker, Artist, Other	stable(customer_id + "occ", 10)
city_category	A, B, C	stable(customer_id + country, 3)
stay_in_current_city_years	0, 1, 2, 3, 4+	stable(customer_id + "stay", 5)
marital_status	Single, Married	stable(customer_id + "mar", 2)

Total customers generated: **4,338 unique customers**. Saved to `customer_master_data.csv`.

3.2 Product Master Data

We take all unique StockCodes from the cleaned data. For each product, we pick the longest non-empty description from all rows that share that StockCode. This avoids keeping a short or partial description when a fuller one exists. If there is no description at all, we set it as UNKNOWN.

The product attributes generated are:

Attribute	Values	Hash Input
product_category	Home Decor, Kitchen, Stationery, Toys, Seasonal Gifts, Bags, Jewelry, Garden, Party, Other	stable(product_id + "cat", 10)
supplier_id	SUP01 to SUP05	stable(product_id, 5)
supplier_name	Northwind Supplies, Atlantic Gifts Ltd, Midlands Wholesale, Celtic Craft Co, EuroPack Partners	Mapped from supplier_id

Total products generated: **3,665 unique products**. Saved to `product_master_data.csv`.

3.3 Splitting Online and Periodic Streams

The assignment says that about 70 percent of the transactions should be treated as online stream data and the remaining 30 percent as periodic store data. The split is applied at the invoice level, not the transaction line level, so all items in a single cart get the same mode.

The rule is:

```
sales_data["sales_mode"] = sales_data["transaction_id"].apply(
    lambda x: "online" if stable(x, 10) < 7 else "periodic"
)
```

If the hash value falls in 0 to 6 (seven out of ten), the transaction is online. If it falls in 7, 8, or 9 (three out of ten), it is periodic.

Stream	File	Transactions
Online	online_sales_stream.csv	278,740
Periodic	periodic_sales_staging.csv	119,144

The periodic data is sorted chronologically by event_time and divided into 48 release groups. Each group represents one simulated hourly batch. This matches the assignment requirement that the physical store sends one batch every hour over a 48-hour simulation.

3.4 Store Assignment

The assignment defines exactly five stores. No additional stores are allowed. The five stores are stored in memory, not in a CSV file:

Store ID	**Store Name**	**City**
1	Online	Web
2	London Bridge	London
3	Manchester Arndale	Manchester
4	Birmingham Bullring	Birmingham
5	Edinburgh Princes	Edinburgh

Online transactions always go to store 1. Periodic (physical store) transactions are assigned to one of stores 2, 3, 4, or 5 using `stable(transaction_id, 4)` to pick the index deterministically.

4. Data Warehouse Design (Star Schema)

The Data Warehouse uses a star schema. There is one central fact table called `transaction_fact` that holds the sales records. Five dimension tables surround it. The schema is defined in `Create-DW/create_dw.sql` and runs against a MySQL database called `retail_dw`.

4.1 Dimension Tables

dim_customer

Stores one row per unique customer. The primary key is an auto-generated `customer_key`. The natural key `customer_id` has a unique constraint. Contains: gender, age_group, occupation, city_category, stay_in_current_city_years, marital_status.

dim_product

Stores one row per unique stock code. Contains: product_id (unique), product_name, product_category. The supplier information is stored in a separate dimension.

dim_supplier

Stores the five suppliers. Contains: supplier_id (unique), supplier_name. Each product is linked to a supplier through the fact table.

dim_store

Stores exactly five rows. Contains: store_id (unique, 1 to 5), store_name, city. This is a fixed table, inserted once and never changed.

dim_time

Covers every single hour from December 1, 2010 to December 31, 2011. That is 9,504 hourly records. Each row contains: full_date, hour, day, month, quarter, year, day_of_week, weekday_weekend, season, half_year. The season and half_year values are derived from the month field:

- Winter: December, January, February
- Spring: March, April, May
- Summer: June, July, August
- Autumn: September, October, November
- Half year 1: January to June
- Half year 2: July to December

4.2 Fact Table

The `transaction_fact` table stores one row per transaction line (one product in one invoice). It has foreign keys to all five dimensions and three measured values: quantity, unit_price, and revenue. Revenue is calculated as quantity multiplied by unit_price. The `sales_mode` column stores whether the transaction came through the online stream or the periodic staging pipeline. The `processing_mode` column is set to "meshjoin" to indicate it was processed through the MESHJOIN algorithm.

```
create table transaction_fact (
    fact_key              int auto_increment primary key,
    transaction_id        varchar(30) not null,
    transaction_line_number int unsigned not null,
    customer_key          int not null,
    product_key           int not null,
    supplier_key          int not null,
    store_key             int not null,
    date_key              int not null,
    quantity              int not null,
    unit_price            decimal(12,2) not null,
    revenue               decimal(14,2) not null,
    sales_mode            varchar(20) not null,
    processing_mode       varchar(20) not null,
    unique (transaction_id, transaction_line_number),
    check (quantity > 0),
    check (unit_price > 0),
    check (sales_mode in ('online', 'periodic'))
);
```

4.3 Staging and Shed Tables

Two additional working tables are part of the schema. `periodic_sales_data` holds the periodic transactions with their release_hour and a released flag, used by the periodic extractor thread to know which batch to push next. The `shed_table` temporarily holds transactions that could not enter the stream buffer during peak load. These are recovered and re-inserted when the buffer has capacity.

5. MESHJOIN Algorithm

MESHJOIN was introduced by Polyzotis in 2008 to solve the problem of joining a high-speed data stream with large master data stored on disk. The core idea is that we cannot load all master data into RAM at once because it is too large, and we cannot query the database for every individual transaction because that would be too slow. MESHJOIN solves this by processing transactions in overlapping batches (chunks) while cycling through master data partitions in the background.

5.1 Configuration Parameters

Parameter	Value
Stream buffer capacity	1,000 transactions
Chunk size (w)	500 transactions per chunk
Partition size	500 tuples per master data page
Customer master data partitions	9 pages (4,338 / 500 = ~9)
Product master data partitions	8 pages (3,665 / 500 = ~8)

5.2 Key Components

Stream Buffer

A Python deque with a maximum size of 1,000. Incoming transactions from the online and periodic threads are pushed here first. The MESHJOIN worker reads from this buffer in chunks of 500. If the buffer is at capacity, incoming periodic transactions are shed to the shed table.

DiskReader Class

This class reads master data CSV files in pages of 500 rows. All pages are loaded at startup into a list of partitions. When `get_next_partition()` is called, it returns the current page and advances the index with wrap-around, so the master data is traversed cyclically:

```
def get_next_partition(self):
    partition = self.partitions[self.current_partition_index]
    self.current_partition_index = (self.current_partition_index + 1) % len(self.partitions)
    return partition
```

Hash Table H

Two dictionaries, one for customer matching and one for product matching. Both are multi-maps: a single key can point to multiple transactions. When a master data page is loaded, we look up each row's ID in the hash

table and enrich all matching transactions.

```
# hash table H maps join attributes to transactions
self.hash_customer = {}    # customer_id -> list of transactions
self.hash_product  = {}    # product_id  -> list of transactions
```

Queue

A deque that tracks chunks in the order they entered. Each chunk has two counters: how many customer partitions it still needs to see, and how many product partitions it still needs to see. A chunk is not removed until both counters reach zero, meaning it has been compared against the complete customer master data and the complete product master data at least once. This guarantees no match is missed.

5.3 The Four-Step Iteration

Each call to `run_one_iteration()` performs four steps in order:

1. **Load a chunk:** Up to 500 transactions are moved from the stream buffer into Hash Table H. Each transaction is indexed by customer_id in `hash_customer` and by product_id in `hash_product`. A node representing this chunk is added to the queue with lifetime counters set to 9 (customer partitions) and 8 (product partitions).
2. **Load a master data partition:** The next customer page and the next product page are both read from DiskReader into the disk buffer.
3. **Probe and enrich:** For every customer tuple in the customer page, the algorithm probes `hash_customer`. If a match is found, it writes the demographic attributes (gender, age group, occupation, etc.) into the matching transaction objects. The same is done for the product page against `hash_product`, adding product name, category, supplier, assigned store, and computed sale value.
4. **Evict completed chunks:** The queue is inspected from the front (oldest chunk). If a chunk has seen all 9 customer pages and all 8 product pages, it is removed from the queue. Its transactions are removed from both hash tables and are sent to the output queue for fact table insertion.

5.4 Data Enrichment Detail

Customer pass adds: gender, age_group, occupation, city_category, stay_in_current_city_years, marital_status.

Product pass adds: product_name, product_category, supplier_id, supplier_name, store_id, and revenue calculated as:

```
tx["sale"] = round(quantity * unit_price, 2)
```

Store assignment is done during the product pass. If the transaction's sales_mode is "online", it is assigned store_id = "1". If it is "periodic", the store is assigned as:

```
store_index = stable(tx["transaction_id"], 4)
tx["store_id"] = ["2", "3", "4", "5"][store_index]
```

5.5 Why Staggered Processing Matters

The core advantage of MESHJOIN is that multiple chunks overlap in memory at the same time. When chunk 1 is being compared against master data pages 2, 3, 4..., chunk 2 has already entered the hash table and is waiting for pages 1, 2, 3... This staggered approach means the master data is traversed once continuously in a cycle, and every chunk eventually sees every page without redundant disk reads or pauses in the stream.

6. Multi-Threading and Load Shedding

The pipeline runs three concurrent threads. These are defined in `threads.py`. All threads share a `threading.Lock()` called `buffer_lock` that controls access to the shared stream buffer and prevents race conditions.

6.1 Thread 1: StreamProducerThread (Online Stream)

This thread reads `online_sales_stream.csv` and pushes records into the stream buffer in batches. It runs continuously until all online records are consumed. If the stream buffer is full, the thread releases the lock briefly and retries, giving the MESHJOIN worker a chance to consume and free up space. It never drops an online transaction.

6.2 Thread 2: PeriodicExtractorThread (Periodic Staging + Load Shedding)

This thread simulates the hourly physical store data extraction. At startup it reads the entire `periodic_sales_staging.csv` and groups records by their `release_hour` value (0 to 47). Then it loops through these 48 groups with a small sleep interval between each one to simulate time passing.

Before releasing each hourly batch, the thread first checks if any previously shed transactions can be recovered. If the stream buffer has free space, shed transactions are pushed back in. Then the current hour's transactions are pushed. If the stream buffer is at capacity, the remaining periodic transactions go to the `shed_queue` in memory instead of the stream buffer.

```
# if buffer has space, push transaction
if len(self.mj.stream_buffer) < self.mj.stream_buffer_size:
    self.mj.add_to_stream_buffer(tx)
    self.total_periodic_pushed += 1
else:
    # buffer full -> shed periodic transaction
    self.shed_queue.append(tx)
    self.total_shed += 1
```

Once all 48 hours are released, the thread keeps trying to push any remaining shed transactions until they are all recovered.

6.3 Thread 3: MeshJoinWorkerThread (MESHJOIN Execution)

This thread runs in a loop and calls `run_one_iteration()` on the MeshJoin operator whenever there are transactions in the stream buffer or chunks still in the queue. Each enriched transaction it produces is appended to the shared `enriched_output` deque, which the main pipeline reads from to build fact records for database insertion.

6.4 Thread Interaction Summary

```
StreamProducerThread ----push----> stream_buffer
PeriodicExtractorThread --push---> stream_buffer
                        --shed----> shed_queue
                        <--recover- shed_queue

MeshJoinWorkerThread ---read-----> stream_buffer
                     ---write----> enriched_output

main.py -----------------read-----> enriched_output
        ---------------write-----> transaction_fact (MySQL)
```

6.5 Execution Results

When the pipeline ran on the full dataset, the execution summary was as follows:

Metric	**Value**
Total execution time	approximately 10 seconds
Online transactions pushed to stream buffer	278,740
Periodic transactions pushed to stream buffer	800
Periodic transactions shed to shed_queue	118,344
Shed transactions recovered	118,344
Total transactions enriched by MESHJOIN	397,884

All 397,884 cleaned transactions were enriched and loaded into the Data Warehouse. No transaction was permanently lost. The shedding and recovery mechanism ensured that even though 118,344 periodic transactions could not enter the stream buffer during the high-load phase of the online stream, they were all recovered afterward and processed successfully.

7. Pipeline Execution and Fact Loading

The main controller is `main.py`. When you run it, it performs the following steps in order.

First it connects to the MySQL database and loads all five dimension tables into in-memory Python dictionaries. This is done so that the fact loader can quickly look up the surrogate key for each dimension without running individual SQL queries for every single transaction. For example, instead of running "SELECT customer_key FROM dim_customer WHERE customer_id = X" for each of 397,884 records, it loads all 4,338 customer keys once into a dictionary and uses dictionary lookup throughout the whole run.

```
cursor.execute("SELECT customer_id, customer_key FROM dim_customer")
customer_keys = {str(row[0]): row[1] for row in cursor.fetchall()}
```

Then it initializes the MeshJoin operator with the two master data CSV paths and starts all three threads. The main thread enters a loop where it reads enriched records from the output queue and builds fact records. For each enriched transaction, it maps the natural IDs to surrogate keys from the cached dictionaries, parses the event_time to find the matching date and hour in `dim_time`, and calculates the final revenue value.

Facts are inserted in batches of 500 using `executemany()` for efficiency. The main loop continues until both producer threads have finished and the MESHJOIN queue is empty.

7.1 How to Run the Full Pipeline

1. Run the schema DDL in MySQL: `mysql -u root -p < Create-DW/create_dw.sql`
2. Generate master data: `python3 Mesh-Join/data_preparation.py`
3. Load dimensions: `python3 Mesh-Join/database.py`
4. Run MESHJOIN and load facts: `python3 Mesh-Join/main.py`
5. Run the analysis queries: `mysql -u root -p -t retail_dw < Queries-DW/analysis_queries.sql > query_results.txt`

8. DW Analysis Queries

Once the fact table is populated, 20 OLAP queries answer business questions about the retail chain. All queries are in `Queries-DW/analysis_queries.sql`. Below is a description of each one.

8.1 Query Descriptions

Query	Purpose	Key Dimensions Used
Q1	Top 5 revenue-generating products on weekdays and weekends, broken down by month	dim_product, dim_time
Q2	Customer spending by gender, age group, and city category	dim_customer
Q3	Product category sales grouped by customer occupation	dim_customer, dim_product
Q4	Revenue trends by gender and age group across all four quarters	dim_customer, dim_time
Q5	Top 5 customer occupations by total revenue per product category	dim_customer, dim_product
Q6	Revenue by city category and marital status for the latest 6 months of data	dim_customer, dim_time
Q7	Average purchase amount by how long a customer has lived in their current city, split by gender	dim_customer
Q8	Total revenue by city category A, B, C for each product category	dim_customer, dim_product
Q9	Month-over-month growth percentage per product category using LAG window function	dim_product, dim_time
Q10	Comparison of weekday sales vs. weekend sales broken down by customer age group	dim_customer, dim_time
Q11	Top 5 products on weekdays vs. weekends, grouped by month	dim_product, dim_time
Q12	Quarter-over-quarter revenue growth rate per store using LAG window function	dim_store, dim_time
Q13	Supplier sales contribution broken down by store and product name	dim_supplier, dim_store, dim_product
Q14	Seasonal sales analysis (Spring, Summer, Autumn, Winter) per product category	dim_product, dim_time
Q15	Month-to-month revenue volatility percentage per store and supplier combination	dim_store, dim_supplier, dim_time

Q16	Top 5 product pairs most frequently bought together in the same cart (product bundling)	dim_product
Q17	Yearly revenue rolled up by store, supplier, and product using the ROLLUP clause	dim_store, dim_supplier, dim_product, dim_time
Q18	Revenue and quantity sold by product for first half of year vs. second half vs. full year	dim_product, dim_time
Q19	Identifies days where a product's daily revenue exceeded double its own average daily revenue (revenue spike detection)	dim_product, dim_time
Q20	Creates and queries the STORE_QUARTERLY_SALES view showing quarterly revenue per store ordered by store name and quarter	dim_store, dim_time

8.2 Notable Techniques Used

Several queries use window functions like `LAG()` to compare the current period against the previous period within the same group. For example, Q9 computes month-over-month growth by comparing each month's product category revenue against the previous month's revenue in the same category:

```
lag(revenue) over (partition by product_category order by year, month)
```

Query 17 uses MySQL's `WITH ROLLUP` modifier on a GROUP BY clause to produce hierarchical subtotals. For example, the query produces rows for individual products, then a subtotal for each supplier within a store, then a total for the store, and finally a grand total. `COALESCE()` is used to replace the NULL values that ROLLUP inserts for aggregated rows with readable labels like "All Stores" or "All Products".

Query 16 joins the fact table with itself on the same transaction_id to find product pairs bought in the same invoice. The condition `f1.product_key < f2.product_key` ensures each pair is counted only once.

Query 20 creates a view called `STORE_QUARTERLY_SALES` that can be queried at any time to get a quick overview of how each store is performing quarter by quarter. The query at the end of the file selects all rows from this view.

9. Conclusion

This project built a complete near-real-time and periodic Data Warehouse for a retail chain from start to finish. The pipeline handles data at every stage: raw data cleaning, deterministic synthetic master data generation, two-stream splitting, MESHJOIN-based data enrichment, multi-threaded loading with load shedding and recovery, and 20 OLAP analytical queries.

The most technically demanding part of the project was implementing the MESHJOIN algorithm correctly. The key challenge is making sure each chunk of transactions stays in memory long enough to be compared against the full customer master data and the full product master data before it is evicted from the hash table. Getting the queue lifetime counters right, and making sure the hash table cleanup does not accidentally remove a transaction before it has completed both passes, required careful coordination between the DiskReader's cyclic traversal and the queue's tracking logic.

The load shedding mechanism was also important. During a real streaming ETL pipeline, the online stream will sometimes produce data faster than MESHJOIN can process it. To handle this without losing periodic transactions, the PeriodicExtractorThread redirects overflow records to the shed_queue and gradually recovers them when the stream buffer is less busy. In the final run, all 118,344 shed transactions were successfully recovered and processed.

The final Data Warehouse contains 397,884 fact records enriched with customer demographics, product attributes, supplier information, store assignments, and computed revenue values. The 20 OLAP queries and the STORE_QUARTERLY_SALES view provide business-level insights into customer behavior, product performance, seasonal trends, and store-level revenue patterns across the full year of retail data.